

Relatório de Auditoria de Segurança

Café com Destino (AmplieChef)

Data: 02/09/2026 · Commit: 136b6d7 (main)

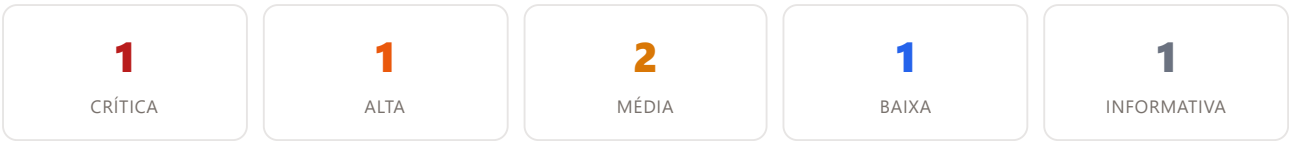
Escopo auditado: aplicação SPA (`src/`), esquema e migrations do banco (`supabase/schema.sql` , `supabase/migrations/0002-0045`), Edge Functions (`supabase/functions/`), artefatos de deploy (`Dockerfile` , `nginx.conf` , `.env*` , `.dockerignore`) e arquivos versionados (`git ls-files` / histórico).

LINGUAGEM / BUILD	TypeScript, Vite 6, React 19 (SPA)
BACKEND / DADOS	Supabase (PostgreSQL 17) — sem servidor de aplicação próprio
ORM / QUERY BUILDER	@supabase/supabase-js (PostgREST) + funções RPC PL/pgSQL
AUTENTICAÇÃO	Supabase Auth (GoTrue) via Edge Function secure-login; sessão em localStorage
AUTORIZAÇÃO	RLS no Postgres + <code>has_any_permission()</code> (coluna <code>profiles.permissions</code>); espelho no front em <code>src/lib/permissions.ts</code>
ISOLAMENTO	Instância única (um restaurante). Fronteira real: anon (cardápio público /pedir) x authenticated (funcionário) x papel/permissão do funcionário
DEPLOY	Dockerfile multi-stage -> nginx:1.27-alpine servindo dist/; EasyPanel injeta VITE_* como build-args
EDGE FUNCTIONS	Deno — secure-login, admin-create-user (usam <code>SUPABASE_SERVICE_ROLE_KEY</code> injetada pelo Supabase)

NOTA METODOLÓGICA — COMO CADA CATEGORIA FOI MAPEADA PARA A STACK

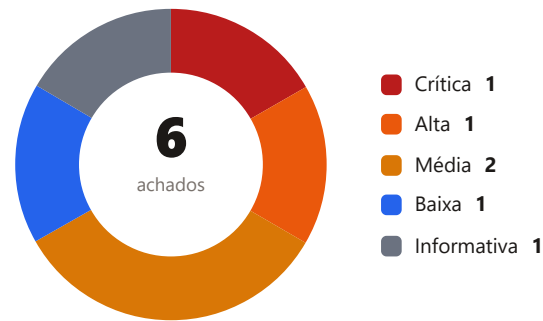
- 1. Banco sem tranca (isolamento):** O projeto e instancia unica, entao "tenant" = a fronteira anon/authenticated/papel. Mapeamos toda policy RLS de SELECT/UPDATE/INSERT/DELETE em `supabase/schema.sql` + migrations 0002-0045 e cruzamos com os pontos de leitura/escrita em `src/context/AppContext.tsx`. Procuramos tabelas cujo SELECT/UPDATE so exige `auth.role() = 'authenticated'` enquanto a UI restringe por permissao.
- 2. Permissão definida no navegador:** Para cada gate `hasPermission()` / tela escondida no front, verificamos se existe verificacao equivalente no servidor — policy RLS `permission-aware`, `has_any_permission()` dentro da RPC, ou trigger. Onde o servidor so checa `authenticated`, e achado.
- 3. IDOR:** Percorremos todos os handlers de escrita de `AppContext.tsx` e todas as RPCs de `supabase/migrations/*`. Como o modelo e instancia unica, "posse" recai sobre papel/permissao e sobre a coluna/estado que a operacao pode tocar. Procuramos escrita direta por ID que ignore a camada RPC endurecida.
- 4. Chaves expostas:** Varredura de `.env*`, `Dockerfile`, `nginx.conf`, `supabase/`, `scripts/`, `docs/`, `historico git` (`git log -- .env`) e arquivos versionados sensiveis (`git ls-files`). Distincao entre chave publica (anon key, por design no bundle) e segredo real.
- 5. Inputs sem tratamento (XSS):** Busca por `dangerouslySetInnerHTML`, `innerHTML`, `v-html`, `eval`, `new Function`, `render` de `markdown/HTML` e `href/src` controlados por usuario em todo `src/`. Backend: input de usuario em HTML de e-mail/template (nao ha e-mail transacional proprio).

Resumo executivo

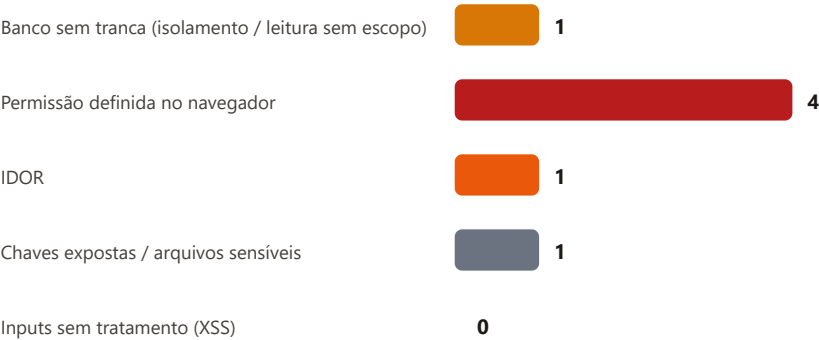


6 achados no total — 5 acionáveis + 1 informativo. Nenhuma falha de XSS e nenhum segredo real no código-fonte (ver Pontos fortes).

Distribuição por severidade



Achados por categoria



Pontos fortes (o que está protegido)

- Validação de preço no servidor.

create_order_and_credit_cash (migrations 0017/0023/0027) recalcula preço unitário, adicionais, subtotal e total a partir de products — ignora o valor enviado pelo cliente. Adicional inexistente ou produto removido derruba a transação inteira.
- Cardápio público (anon) fortemente contido.

Migration 0023: chamador anon tem canal forçado a online, itens limitados a 1-50, nome/telefone do cliente validados em tamanho e formato, desconto e already_paid zerados, notes <= 1000. anon só tem EXECUTE em create_order_and_credit_cash — nenhum SELECT em orders.
- Escalonamento de privilégio em perfis bloqueado no banco.

Trigger prevent_self_privilege_escalation (0014/0015) rejeita alteração de role/active/code/permissions por quem não é admin, independente da policy. admin-create-user (Edge Function) confere o papel do chamador pelo próprio JWT antes de usar a service role.
- Grants de função travados.

Migration 0016 revoga EXECUTE de PUBLIC/anon em todas as funções do schema public e re-concede só as necessárias, a authenticated. alter default privileges ... revoke execute fecha o padrão para funções futuras.
- Camada de caixa reescrita como RPC security definir com guard de permissão.

Migrations 0027/0028: open_cash_shift, close_cash_shift, add_cash_movement, credit_partial_payment, reverse_partial_payment, reverse_paid_order, record_cash_expense — todas checam has_any_permission(...) e resolvem o autor por auth.uid(). Escrita direta em cash_shifts/cash_movements revogada de authenticated/anon.

Livro-caixa e trilha de auditoria imutáveis. `cash_ledger` e `audit_log` (0025) são append-only: RLS sem policy de INSERT/UPDATE/DELETE, revoke de authenticated/anon/public e triggers `reject_mutation` que barram até `service_role/console`.

PIN de caixa com hash e rate-limit. Migration 0042: `profiles.code` migrado para `bcrypt` (`crypt+gen_salt(bf)`), trigger que hasheia toda escrita nova, validação server-side (`validate_user_pin/validate_manager_pin`) devolvendo só boolean. `pin_attempts` faz logout. Colunas `code` (0028) e `cpf` (0043) revogadas do cliente e retiradas do realtime.

Força bruta de login contida. `secure-login` (Edge Function) centraliza o login, com logout de 5 tentativas / 15 min por e-mail em `login_attempts` (RLS sem policy). Mensagens de erro não permitem enumeração de conta. Formato de e-mail/senha validado antes de bater no GoTrue.

RLS permission-aware onde foi aplicada. Migration 0022: `products`, `dining_tables`, `ingredients`, `suppliers`, `categories`, `table_sectors` etc. têm INSERT/UPDATE/DELETE amarrados a `has_any_permission()` com as mesmas chaves do front. `audit_log` e `financial_entries` também nos SELECT.

RPCs de comanda atômicas e com permissão. Migration 0037: `comanda_add_items`, `comanda_cancel_item`, `comanda_set_charge`, `comanda_transfer` — FOR UPDATE na mesa, checagem de `has_any_permission` e ordem estável de lock para evitar deadlock. `adjust_stock` (0039) idem para entrada/perda/cortesia.

Sem superfície de XSS no frontend. Nenhum `dangerouslySetInnerHTML`, `innerHTML`, `eval`, `new Function` ou `render` de `markdown/HTML` em todo `src/`. React 19 escapa o texto interpolado. Os poucos `href` dinâmicos (`getWhatsAppLink`, `SupportButton`) têm prefixo `https://` fixo e só concatenam dígitos de telefone / texto `encodeURIComponent`. `primary_color` não é injetado em `style` em runtime.

Sem segredo real no repositório. `git ls-files` e `git log -- .env` não revelam `service_role_key`, `JWT secret`, senha de banco ou chave privada. `.env` está no `.gitignore`; o `VITE_SUPABASE_ANON_KEY` no `Dockerfile/.env` é a chave pública anon (por design no bundle). Edge Functions recebem `SUPABASE_SERVICE_ROLE_KEY` via secret injetado pelo Supabase, não do código.

Pontos fracos (riscos centrais)

Camada de autorização em duas velocidades. A escrita passou por um endurecimento sério (RPCs security definir com `has_any_permission`, triggers de imutabilidade, validação de preço no servidor), mas sobraram **ilhas que só checam** `auth.role() = 'authenticated'`: a escrita de `company_profile` (Achado 1), o UPDATE de `orders` (Achado 2) e duas RPCs de venda sem guard (Achado 4). Como o front esconde essas ações por permissão, a diferença só aparece para quem chama a API direto.

Leitura sem escopo de papel. Quase todos os SELECT financeiros e a PII de clientes em `orders.customer` são visíveis a qualquer funcionário logado (Achado 3). `audit_log` e `financial_entries` mostram que o padrão permission-aware era conhecido — só não foi aplicado de forma consistente.

Storage sem dono. As policies do bucket `product-images` não amarram update/delete ao autor nem a permissão (Achado 5) — qualquer conta apaga a mídia do cardápio público.

Achados detalhados

Crítica (1)

#1 · Tabela company_profile pode ser reescrita por qualquer funcionário autenticado **CRÍTICA**

Categoria 2 — Permissão definida no navegador

Arquivo:linha supabase/migrations/0018_public_online_ordering.sql:71-72
 supabase/migrations/0026_service_fee_couvert_discount_finance.sql:14-21
 src/context/AppContext.tsx:512-517

Por que é explorável A única policy de UPDATE de company_profile exige apenas auth.role() = 'authenticated' — qualquer usuário logado (garçom, cozinha, estoque) passa. Não há trigger de proteção de coluna (ao contrário de profiles, que tem prevent_self_privilege_escalation). A migration 0022 tornou dezenas de tabelas permission-aware, mas company_profile ficou de fora e nunca foi corrigida (verificado até a 0045). discount_limits não tem sequer CHECK constraint.

Impacto encadeado — a linha guarda parâmetros que o servidor confia:

- discount_limits é lida por discount_limit_percent() para autorizar desconto acima do teto (0027:307-316). Um caixa faz update company_profile set discount_limits='{ "caixa":100 }' e passa a dar 100% de desconto sem PIN de gerente.
- blind_conference_threshold define quando a diferença de fechamento de caixa exige justificativa (0027:599). Eleva-la esconde furto de caixa.
- pix_key e bank_info aparecem no cardápio público (/pedir). Troca-las redireciona o pagamento dos clientes.
- buffet_prices (preço por kg) alimenta o preço de venda dos itens por quilo no PDV.
- min_order_value / delivery_fee afetam todo pedido online.

Explorável direto pelo console do navegador com a sessão de qualquer funcionário — não depende da UI.

EVIDÊNCIA

```
-- 0018_public_online_ordering.sql:71
create policy "authenticated_update_company_profile" on company_profile for update
  using (auth.role() = 'authenticated') with check (auth.role() = 'authenticated');

-- 0026: colunas de negócio que passam a morar nessa mesma linha, sem CHECK de autor:
-- discount_limits jsonb default '{"caixa":5,"gerente":20,"financeiro":10,"admin":100}'
-- blind_conference_threshold, service_fee_percent, couvert_value
-- buffet_prices, pix_key, bank_info, fiscal_info, min_order_value, delivery_fee

// src/context/AppContext.tsx:514 (UI gateada por 'empresa.editar_perfil' / 'empresa.editar_precos_buffet')
supabase.from('company_profile').update(toRow(profile)).eq('id', true)
```

CORREÇÃO SUGERIDA

```
Substituir a policy por uma permission-aware:
create policy company_profile_update on public.company_profile for update
  using
  (public.has_any_permission(array['empresa.editar_perfil','empresa.editar_midia','empresa.editar_precos_buffet']))
  with check
  (public.has_any_permission(array['empresa.editar_perfil','empresa.editar_midia','empresa.editar_precos_buffet']));

Isolar os campos financeiros sensíveis (discount_limits, blind_conference_threshold, pix_key,
bank_info, fiscal_info) numa RPC security_definer só-admin, com linha em audit_log.
Adicionar CHECK em discount_limits (cada valor entre 0 e 100).
```

Alta (1)

#2 · UPDATE irrestrito em orders permite adulterar pagamento, valores e dados do cliente ALTA

Categoria 2 — Permissão definida no navegador · Categoria 3 — IDOR

Arquivo:linha `supabase/migrations/0022_permission_aware_rls.sql:46-55`
`src/context/AppContext.tsx:1083-1108`

Por que é explorável online_menu.acessar e concedida a garcom, caixa e gerente; kitchen.avancar_status a cozinha. Qualquer um desses papéis pode fazer, direto pelo supabase-js:
`update orders set payment_status='pagamento_aprovado', order_status='concluido' where id='<qualquer>'`
`update orders set total=1, discount=0, nfce_key='...', customer='{...}' where id='<qualquer>'`

Isso contorna toda a camada RPC endurecida (0017/0023/0027), que so protege a criacao e o estorno. O estorno legitimo (reverse_paid_order) exige caixas.estornar_venda + PIN de gerente; aqui um garcom marca `payment_status='pagamento_estornado'` num pedido pago sem nada disso, ou quita um pedido de entrega ainda nao pago, ou reescreve o total de pedidos historicos para encobrir diferenca de caixa. Relatorios e conferencia leem orders diretamente.

EVIDÊNCIA

```
-- 0022_permission_aware_rls.sql:46
create policy permission_update_orders on public.orders
  for update
  using (public.has_any_permission(array[
    'kitchen.avancar_status', 'entregas.despachar', 'vendas.emitir_nfce',
    'vendas.acessar', 'online_menu.acessar'
  ]))
  with check (public.has_any_permission(array[ ... as mesmas ... ]));
-- sem restricao de coluna, sem restricao de estado, sem checar autoria (waiter_name)
-- 'orders' so tem trigger BEFORE INSERT (0036, order_number). Nada protege o UPDATE.
```

CORREÇÃO SUGERIDA

Restringir a policy de UPDATE ao minimo do fluxo de cozinha/expedicao e mover o resto para RPC:

- Manter UPDATE direto apenas de order_status / prepared_at / delivered_at / delivery_driver_name, idealmente via RPC `advance_order_status(p_id, p_status)` que valida a transicao de estado.
- payment_status, payment_method, total, subtotal, discount, fiscal_issued, nfce_key, customer, split_payments, shift_id: nunca por UPDATE direto do cliente – so RPC security definer com checagem de permissao especifica.
- Trigger BEFORE UPDATE em orders que rejeita alteracao das colunas financeiras quando `auth.uid()` nao tem vendas.acessar (cinto e suspensorio).

Média (2)

#3 · Livro-caixa, turnos, movimentações e PII de clientes legíveis por qualquer funcionário MÉDIA

Categoria 1 — Banco sem tranca (isolamento / leitura sem escopo)

Arquivo:linha `supabase/migrations/0025_cash_ledger_audit_pin.sql:73-74`
`supabase/schema.sql:276`
`supabase/migrations/0022_permission_aware_rls.sql:43-44, 60-61, 66-67`
`supabase/migrations/0038_loss_records_courtesy_printers_drivers.sql:95-96, 102-103`

Por que é explorável

A UI esconde Livro-Caixa (livro_caixa.acessar), Caixas (caixas.acessar) e os endereços de entrega atrás de permissões que cozinha/estoque/garcom não tem. Mas o SELECT dessas tabelas no Postgres só exige `authenticated`. Um `select * from cash_ledger / cash_shifts / cash_movements / orders` feito pelo supabase-js de qualquer sessão de funcionário devolve tudo:

- livro-caixa completo (cada venda, adiantamento, sangria, despesa, com motivo e autor);
- diferença de fechamento de todos os turnos;
- nome, telefone e endereço residencial de todos os clientes de delivery — exposição de dados pessoais (LGPD).

O risco cresce se um terminal de baixo privilégio (tablet da cozinha) for comprometido. `audit_log` e `financial_entries` já foram feitos `permission-aware` nas mesmas migrations — o padrão era conhecido, só não foi aplicado de forma consistente.

Observação relacionada: `secure-login` não verifica `profiles.active`; um funcionário desativado ainda recebe tokens e mantém esse mesmo SELECT amplo (as escritas via `has_any_permission` já checam `active`).

EVIDÊNCIA

```
-- 0025:73 cash_ledger (livro-caixa append-only: TODA transacao com valor/forma/motivo)
create policy cash_ledger_select on public.cash_ledger
  for select using (auth.role() = 'authenticated');

-- schema.sql:276 cash_shifts (fundo, vendas por forma, diferença de fechamento, notas)
create policy "authenticated_all_cash_shifts" ... using (auth.role() = 'authenticated');
-- 0022:66 authenticated_select_cash_shifts      → idem
-- 0022:60 authenticated_select_cash_movements   → sangria/reforço, valores, responsável
-- 0022:43 authenticated_select_orders           → orders.customer = {name, phone, address}
-- 0038:95/102 loss_records / courtesy_records  → idem (auth.role())

-- Contraste (mesma migration 0025/0026 fez certo):
-- 0025:114 audit_log_select                     using has_any_permission(['auditoria.acessar'])
-- 0026:81 financial_entries_select              using has_any_permission(['financeiro_dre.acessar'])
```

CORREÇÃO SUGERIDA

```
Tornar os SELECT permission-aware, como já foi feito para audit_log/financial_entries:
cash_ledger      → using has_any_permission(array['livro_caixa.acessar', 'caixas.acessar'])
cash_shifts      → using has_any_permission(array['caixas.acessar'])
cash_movements   → using has_any_permission(array['caixas.acessar'])
loss_records / courtesy_records → using has_any_permission(array['estoque.acessar', ...])
Para orders: expor a coluna customer (PII) só a quem tem vendas.acessar / entregas.acessar (view ou mascaramento).
Fazer secure-login recusar login de conta active = false.
```

#4 · RPCs `create_order_and_credit_cash` e `close_comanda_and_pay` não verificam permissão de venda do chamador MÉDIA

Categoria 2 — Permissão definida no navegador

Arquivo:linha `supabase/migrations/0027_cash_rpc_cutover.sql:169-183, 416-428, 1065-1069`

Por que é explorável

Ambas rodam como dona da função (ignoram RLS) e estão liberadas para `authenticated`. Não checam se o chamador tem `pdv.finalizar_venda / mesas.fechar_comanda`. Um funcionário cozinha ou estoque — que a UI nunca deixa vender — pode chamar `supabase.rpc('create_order_and_credit_cash', {...})` e:

- criar pedidos arbitrários (com preço recalculado no servidor, então não é fraude de preço, mas é registro de venda falso);
- injetar valores em `cash_shifts.sales_cash/card/pix...` e no `cash_ledger` do turno aberto, sujando a conferência de caixa.

A validação de preço/estoque de 0017/0023 mitiga o pior caso (venda de R\$500 por R\$5), mas a ausência do guard de permissão que todas as RPCs vizinhas têm e uma inconsistência explorável — inclusive para poluição de dados e negação de conferência.

EVIDÊNCIA

```
-- 0027: todas as outras RPCs de caixa comecam com um guard de permissao ...
open_cash_shift    → if not has_any_permission(array['caixas.abrir'])      then raise ...
add_cash_movement  → if not has_any_permission(array['caixas.movimentacao']) then raise ...
close_cash_shift   → if not has_any_permission(array['caixas.fechar'])     then raise ...
credit_partial_payment / reverse_partial_payment / reverse_paid_order / record_cash_expense → idem

-- ...mas estas duas NAO tem guard nenhum, e sao security definer:
create or replace function public.create_order_and_credit_cash(...) ... security definer
create or replace function public.close_comanda_and_pay(...) ... security definer
grant execute on function public.create_order_and_credit_cash(...) to authenticated, anon; -- 1065
grant execute on function public.close_comanda_and_pay(...) to authenticated; -- 1068
```

CORREÇÃO SUGERIDA

```
Adicionar, no inicio de ambas (para o ramo nao-anonimo):
if auth.role() <> 'anon' and not public.has_any_permission(array[
    'pdv.finalizar_venda','mesas.fechar_comanda','online_menu.finalizar_pedido']) then
    raise exception 'Sem permissao para registrar venda.';
end if;
Manter o ramo anon (cardapio publico) como esta - ja e fortemente validado.
```

Baixa (1)

#5 · Bucket product-images: escrita e exclusão sem escopo de dono nem permissão

BAIXA

Categoria 2 — Permissão definida no navegador

Arquivo:linha `src/lib/storage.ts:54-64`
`DEV_DB_SETUP.md:bloco SQL do bucket`

Por que é explorável

Qualquer funcionario autenticado pode sobrescrever ou apagar todos os objetos do bucket: fotos de produto, logo da empresa e capa do cardapio online (company_profile.logo_url / cover_url, que aparecem na pagina publica /pedir). Resultado: defacement do cardapio publico e perda de midia, feito por conta de baixo privilegio. A UI so oferece gestao de imagem a quem tem produtos.editar / empresa.editar_midia. O bucket e publico para leitura (por design), entao o dano e de integridade/disponibilidade.

EVIDÊNCIA

```
create policy "product_images_authenticated_write" on storage.objects
  for insert to authenticated with check (bucket_id = 'product-images');
create policy "product_images_authenticated_update" on storage.objects
  for update to authenticated using (bucket_id = 'product-images');
create policy "product_images_authenticated_delete" on storage.objects
  for delete to authenticated using (bucket_id = 'product-images');
-- sem owner = auth.uid(), sem has_any_permission(...)
```

CORREÇÃO SUGERIDA

Escopar update/delete por dono (owner = auth.uid()) e/ou por permissao.
 Para a midia institucional (logo/capa), usar prefixo proprio (empresa/) e exigir empresa.editar_midia;
 para produtos/, exigir produtos.editar ou produtos.criar. O INSERT amplo pode continuar.

Informativa (1)

#6 · Diretório supabase/.temp/ versionado (metadados de projeto e string de conexão do pooler)

INFORMATIVA

Categoria 4 — Chaves expostas / arquivos sensíveis

Arquivo:linha supabase/.temp/pooler-url:1
supabase/.temp/linked-project.json:1
.gitignore:9

Por que é explorável Nao ha segredo real aqui: a pooler-url traz so usuario e host (sem senha), o project ref ja vai embutido em VITE_SUPABASE_URL no bundle, e as versoes de componentes sao publicas. Mesmo assim, expoe organizacao, nome do projeto de dev e a topologia do banco sem necessidade, e indica que o .gitignore nao esta sendo respeitado para esse caminho (risco de, no futuro, entrar ali um arquivo que de fato seja sensivel). Classificado como informativo.

EVIDÊNCIA

```
# git ls-files inclui:
supabase/.temp/pooler-url      → postgresql://postgres.vxgkslytnofxtkhjpkxb@aws-0-us-west-
2.pooler.supabase.com:5432/postgres
supabase/.temp/linked-project.json →
{"ref":"vxgkslytnofxtkhjpkxb","name":"ampliechef_dev","organization_id":"ikrbfkaluvaemxzlrhbi", ... }
supabase/.temp/project-ref, gotrue-version, postgres-version, storage-version, ...
# .gitignore tem ".temp/" (linha 9), mas os arquivos foram adicionados antes da regra e continuam rastreados
```

CORREÇÃO SUGERIDA

```
git rm -r --cached supabase/.temp && git commit.
Confirmar que .gitignore cobre supabase/.temp/ (a regra .temp/ ja cobre; o problema e so o index/historico).
Revisar historico: git log --all -- supabase/.temp para garantir que nunca houve um .temp com access-token.
```

Tabela de achados por categoria

Categoria 1 — Banco sem tranca (isolamento / leitura sem escopo)

SEVERIDADE	ARQUIVO:LINHA	DESCRIÇÃO
MÉDIA	supabase/migrations/0025_cash_ledger_audit_pin.sql:73-74 supabase/schema.sql:276 supabase/migrations/0022_permission_aware_rls.sql:43-44, 60-61, 66-67 supabase/migrations/0038_losses_courtesies_printers_drivers.sql:95-96, 102-103	#3 Livro-caixa, turnos, movimentações e PII de clientes legíveis por qualquer funcionário

Categoria 2 — Permissão definida no navegador

SEVERIDADE	ARQUIVO:LINHA	DESCRIÇÃO
CRÍTICA	supabase/migrations/0018_public_online_ordering.sql:71-72 supabase/migrations/0026_service_fee_couvert_discount_finance.sql:14-21 src/context/AppContext.tsx:512-517	#1 Tabela company_profile pode ser reescrita por qualquer funcionário autenticado
ALTA	supabase/migrations/0022_permission_aware_rls.sql:46-55 src/context/AppContext.tsx:1083-1108	#2 UPDATE irrestrito em orders permite adulterar pagamento, valores e dados do cliente
MÉDIA	supabase/migrations/0027_cash_rpc_cutover.sql:169-183, 416-428, 1065-1069	#4 RPCs create_order_and_credit_cash e close_comanda_and_pay não verificam permissão de venda do chamador

SEVERIDADE	ARQUIVO:LINHA	DESCRIÇÃO
BAIXA	src/lib/storage.ts:54-64 DEV_DB_SETUP.md:bloco SQL do bucket	#5 Bucket product-images: escrita e exclusão sem escopo de dono nem permissão

Categoria 3 — IDOR

SEVERIDADE	ARQUIVO:LINHA	DESCRIÇÃO
ALTA	supabase/migrations/0022_permission_aware_rls.sql:46-55 src/context/AppContext.tsx:1083-1108	#2 UPDATE irrestrito em orders permite adulterar pagamento, valores e dados do cliente

Categoria 4 — Chaves expostas / arquivos sensíveis

SEVERIDADE	ARQUIVO:LINHA	DESCRIÇÃO
INFORMATIVA	supabase/.temp/pooler-url:1 supabase/.temp/linked-project.json:1 .gitignore:9	#6 Diretório supabase/.temp/ versionado (metadados de projeto e string de conexão do pooler)

Categoria 5 — Inputs sem tratamento (XSS)

Sem achados nesta categoria — o frontend não possui sinks de HTML/JS dinâmico (verificado).

Recomendações priorizadas

PRIO	SEV.	AÇÃO	DETALHE
P1	CRÍTICA	Fechar a escrita de company_profile	Policy permission-aware + trigger de coluna + isolar campos financeiros (discount_limits, blind_conference_threshold, pix_key, bank_info, fiscal_info) em RPC admin-only com auditoria. (Achado 1)
P1	ALTA	Restringir UPDATE de orders	Limitar a policy às colunas de status de preparo; mover pagamento/valores/fiscal/cliente para RPC security definir com permissão; trigger BEFORE UPDATE nas colunas financeiras. (Achado 2)
P2	MÉDIA	Tornar os SELECT financeiros permission-aware	cash_ledger, cash_shifts, cash_movements, loss_records, courtesy_records; mascarar orders.customer (PII/LGPD); secure-login recusar active = false. (Achado 3)
P2	MÉDIA	Adicionar guard de permissão nas RPCs de venda	create_order_and_credit_cash e close_comanda_and_pay: exigir pdv.finalizar_venda / mesas.fechar_comanda no ramo não-anônimo. (Achado 4)
P3	BAIXA	Escopar as policies do bucket product-images	update/delete por owner = auth.uid() e/ou por permissão; separar prefixo empresa/ (institucional) de produtos/. (Achado 5)
P3	INFORMATIVA	Despoluir o versionamento	git rm -r --cached supabase/.temp; auditar histórico por qualquer .temp/token já commitado. (Achado 6)
P3	INFORMATIVA	Endurecimento contínuo	Revisar CORS * das Edge Functions para uma allowlist de origens; verificação de startup que rejeite SUPABASE_SERVICE_ROLE_KEY ausente/placeholder; considerar rotação periódica da anon key como higiene.

Issues para o GitHub

Um bloco por achado, pronto para copiar e colar. Achados triviais do mesmo tema estão agrupados.

```

--- ISSUE 1 ---

## [Segurança] Tabela company_profile pode ser reescrita por qualquer funcionário autenticado

**Labels:** security, critica
**Classificação:** Crítica · Categoria 2 (Permissão definida no navegador)

### Problema e por que é explorável
A unica policy de UPDATE de company_profile exige apenas auth.role() = 'authenticated' – qualquer usuario logado (garcom, cozinha, estoque) passa. Nao ha trigger de protecao de coluna (ao contrario de profiles, que tem prevent_self_privilege_escalation). A migration 0022 tornou dezenas de tabelas permission-aware, mas company_profile ficou de fora e nunca foi corrigida (verificado ate a 0045). discount_limits nao tem sequer CHECK constraint.

Impacto encadeado – a linha guarda parametros que o servidor confia:
- discount_limits e lida por discount_limit_percent() para autorizar desconto acima do teto (0027:307-316). Um caixa faz update company_profile set discount_limits='{ "caixa":100}' e passa a dar 100% de desconto sem PIN de gerente.
- blind_conference_threshold define quando a diferenca de fechamento de caixa exige justificativa (0027:599). Eleva-la esconde furto de caixa.
- pix_key e bank_info aparecem no cardapio publico (/pedir). Troca-las redireciona o pagamento dos clientes.
- buffet_prices (preco por kg) alimenta o preco de venda dos itens por quilo no PDV.
- min_order_value / delivery_fee afetam todo pedido online.

Exploravel direto pelo console do navegador com a sessao de qualquer funcionario – nao depende da UI.

### Evidência
- `supabase/migrations/0018_public_online_ordering.sql:71-72`
- `supabase/migrations/0026_service_fee_couvert_discount_finance.sql:14-21`
- `src/context/AppContext.tsx:512-517`

...

-- 0018_public_online_ordering.sql:71
create policy "authenticated_update_company_profile" on company_profile for update
  using (auth.role() = 'authenticated') with check (auth.role() = 'authenticated');

-- 0026: colunas de negocio que passam a morar nessa mesma linha, sem CHECK de autor:
-- discount_limits jsonb default '{"caixa":5,"gerente":20,"financeiro":10,"admin":100}'
-- blind_conference_threshold, service_fee_percent, couvert_value
-- buffet_prices, pix_key, bank_info, fiscal_info, min_order_value, delivery_fee

// src/context/AppContext.tsx:514 (UI gateada por 'empresa.editar_perfil' / 'empresa.editar_precos_buffet')
supabase.from('company_profile').update(toRow(profile)).eq('id', true)
...

### Impacto
Caixa/garcom/cozinha altera teto de desconto, limiar de conferência de caixa, chave PIX do cardápio público e preço por kg – fraude financeira e desvio de pagamento de clientes.

### Sugestão de correção
Substituir a policy por uma permission-aware:
  create policy company_profile_update on public.company_profile for update
    using (public.has_any_permission(array['empresa.editar_perfil','empresa.editar_midia','empresa.editar_precos_buffet']))
    with check (public.has_any_permission(array['empresa.editar_perfil','empresa.editar_midia','empresa.editar_precos_buffet']));

Isolar os campos financeiros sensiveis (discount_limits, blind_conference_threshold, pix_key, bank_info, fiscal_info) numa RPC security definir so-admin, com linha em audit_log.
Adicionar CHECK em discount_limits (cada valor entre 0 e 100).

### Critérios de aceite
- [ ] Funcionário sem empresa.editar_perfil recebe erro de RLS ao tentar update company_profile via supabase-js
- [ ] discount_limits, blind_conference_threshold, pix_key, bank_info só alteráveis por admin (ou RPC dedicada)
- [ ] Toda alteração de company_profile gera linha em audit_log com actor_id
- [ ] CHECK constraint valida cada valor de discount_limits no intervalo 0-100
- [ ] Teste de regressão: caixa tenta desconto 100% após adulterar discount_limits -> bloqueado

--- FIM ISSUE 1 ---

```

--- ISSUE 2 ---

[Segurança] UPDATE irrestrito em orders permite adulterar pagamento, valores e dados do cliente

****Labels:**** security, alta

****Classificação:**** Alta · Categoria 2 (Permissão definida no navegador) + Categoria 3 (IDOR)

Problema e por que é explorável

online_menu.acessar e concedida a garcom, caixa e gerente; kitchen.avancar_status a cozinha. Qualquer um desses papeis pode fazer, direto pelo supabase-js:

```
update orders set payment_status='pagamento_aprovado', order_status='concluido' where id='<qualquer>'
update orders set total=1, discount=0, nfce_key='...', customer='{...}' where id='<qualquer>'
```

Isso contorna toda a camada RPC endurecida (0017/0023/0027), que so protege a criacao e o estorno. O estorno legitimo (reverse_paid_order) exige caixas.estornar_venda + PIN de gerente; aqui um garcom marca payment_status='pagamento_estornado' num pedido pago sem nada disso, ou quita um pedido de entrega ainda nao pago, ou reescreve o total de pedidos historicos para encobrir diferenca de caixa. Relatorios e conferencia leem orders diretamente.

Evidência

```
- `supabase/migrations/0022_permission_aware_ri.s.sql:46-55`
- `src/context/AppContext.tsx:1083-1108`
```

...

```
-- 0022_permission_aware_ri.s.sql:46
create policy permission_update_orders on public.orders
for update
using (public.has_any_permission(array[
  'kitchen.avancar_status', 'entregas.despachar', 'vendas.emitir_nfce',
  'vendas.acessar', 'online_menu.acessar'
]))
with check (public.has_any_permission(array[ ...as mesmas... ]));
-- sem restricao de coluna, sem restricao de estado, sem checar autoria (waiter_name)
-- 'orders' so tem trigger BEFORE INSERT (0036, order_number). Nada protege o UPDATE.
...
```

Impacto

Funcionário de baixo privilégio marca pedidos como pagos/estornados e reescreve totais e dados fiscais de qualquer pedido – perda financeira e adulteração de nota.

Sugestão de correção

Restringir a policy de UPDATE ao minimo do fluxo de cozinha/expedicao e mover o resto para RPC:

- Manter UPDATE direto apenas de order_status / prepared_at / delivered_at / delivery_driver_name, idealmente via RPC advance_order_status(p_id, p_status) que valida a transicao de estado.
- payment_status, payment_method, total, subtotal, discount, fiscal_issued, nfce_key, customer, split_payments, shift_id: nunca por UPDATE direto do cliente – so RPC security definir com checagem de permissao especifica.
- Trigger BEFORE UPDATE em orders que rejeita alteracao das colunas financeiras quando auth.uid() nao tem vendas.acessar (cinto e suspensorio).

Critérios de aceite

- [] Garcom não consegue alterar payment_status/total/nfce_key de nenhum pedido via supabase-js
- [] Avançar status de preparo continua funcionando para cozinha/expedição
- [] Marcar pedido como pago/estornado exige RPC dedicada com permissão + (no estorno) PIN de gerente
- [] Trigger BEFORE UPDATE cobre as colunas financeiras de orders
- [] Teste: cozinha tenta update orders set total=1 -> erro

--- FIM ISSUE 2 ---

--- ISSUE 3 ---

[Segurança] Livro-caixa, turnos, movimentações e PII de clientes legíveis por qualquer funcionário

****Labels:**** security, media

****Classificação:**** Média · Categoria 1 (Banco sem tranca (isolamento / leitura sem escopo))

Problema e por que é explorável

A UI esconde Livro-Caixa (livro_caixa.acessar), Caixas (caixas.acessar) e os endereços de entrega atras de permissões que cozinha/estoque/garcom não tem. Mas o SELECT dessas tabelas no Postgres só exige authenticated. Um select * from cash_ledger / cash_shifts / cash_movements / orders feito pelo supabase-js de qualquer sessão de funcionário devolve tudo:

- livro-caixa completo (cada venda, adiantamento, sangria, despesa, com motivo e autor);
- diferença de fechamento de todos os turnos;
- nome, telefone e endereço residencial de todos os clientes de delivery – exposição de dados pessoais (LGPD).

O risco cresce se um terminal de baixo privilégio (tablet da cozinha) for comprometido. audit_log e financial_entries já foram feitos permission-aware nas mesmas migrations – o padrão era conhecido, so não foi aplicado de forma consistente. Observação relacionada: secure-login não verifica profiles.active; um funcionário desativado ainda recebe tokens e mantém esse mesmo SELECT amplo (as escritas via has_any_permission já checam active).

Evidência

```
- `supabase/migrations/0025_cash_ledger_audit_pin.sql:73-74`
- `supabase/schema.sql:276`
- `supabase/migrations/0022_permission_aware_rls.sql:43-44, 60-61, 66-67`
- `supabase/migrations/0038_losses_courtesies_printers_drivers.sql:95-96, 102-103`

...

-- 0025:73 cash_ledger (livro-caixa append-only: TODA transação com valor/forma/motivo)
create policy cash_ledger_select on public.cash_ledger
  for select using (auth.role() = 'authenticated');

-- schema.sql:276 cash_shifts (fundo, vendas por forma, diferença de fechamento, notas)
create policy "authenticated_all_cash_shifts" ... using (auth.role() = 'authenticated');
-- 0022:66 authenticated_select_cash_shifts -> idem
-- 0022:60 authenticated_select_cash_movements -> sangria/reforço, valores, responsável
-- 0022:43 authenticated_select_orders -> orders.customer = {name, phone, address}
-- 0038:95/102 loss_records / courtesies_records -> idem (auth.role())

-- Contraste (mesma migration 0025/0026 fez certo):
-- 0025:114 audit_log_select using has_any_permission(['auditoria.acessar'])
-- 0026:81 financial_entries_select using has_any_permission(['financeiro_dre.acessar'])
...
```

Impacto

Qualquer conta de funcionário lê o livro-caixa inteiro, a diferença de todos os fechamentos e o endereço residencial de todos os clientes de delivery.

Sugestão de correção

Tornar os SELECT permission-aware, como já foi feito para audit_log/financial_entries:

```
cash_ledger -> using has_any_permission(array['livro_caixa.acessar','caixas.acessar'])
cash_shifts -> using has_any_permission(array['caixas.acessar'])
cash_movements -> using has_any_permission(array['caixas.acessar'])
loss_records / courtesies_records -> using has_any_permission(array['estoque.acessar', ...])
```

Para orders: expor a coluna customer (PII) só a quem tem vendas.acessar / entregas.acessar (view ou mascaramento).

Fazer secure-login recusar login de conta active = false.

Critérios de aceite

- [] Sessão de cozinha recebe 0 linhas em select * from cash_ledger / cash_shifts / cash_movements
- [] Endereço/telefone do cliente em orders.customer só visível para papéis com vendas.acessar ou entregas.acessar
- [] secure-login retorna 403 para usuário active = false
- [] Telas Livro-Caixa e Caixas continuam funcionando para caixa/gerente/financeiro

--- FIM ISSUE 3 ---

--- ISSUE 4 ---

[Segurança] RPCs create_order_and_credit_cash e close_comanda_and_pay não verificam permissão de venda do chamador

****Labels:**** security, media

****Classificação:**** Média · Categoria 2 (Permissão definida no navegador)

Problema e por que é explorável

Ambas rodam como dona da funcao (ignoram RLS) e estao liberadas a authenticated. Nao checam se o chamador tem pdv.finalizar_venda / mesas.fechar_comanda. Um funcionario cozinha ou estoque – que a UI nunca deixa vender – pode chamar supabase.rpc('create_order_and_credit_cash', {...}) e:

- criar pedidos arbitrarios (com preco recalculado no servidor, entao nao e fraude de preco, mas e registro de venda falso);
- injetar valores em cash_shifts.sales_cash/card/pix... e no cash_ledger do turno aberto, sujando a conferencia de caixa.

A validacao de preco/estoque de 0017/0023 mitiga o pior caso (venda de R\$500 por R\$5), mas a ausencia do guard de permissao que todas as RPCs vizinhas tem e uma inconsistencia exploravel – inclusive para poluicao de dados e negacao de conferencia.

Evidência

- `supabase/migrations/0027\_cash\_rpc\_cutover.sql:169-183, 416-428, 1065-1069`

...

-- 0027: todas as outras RPCs de caixa comecam com um guard de permissao...

```
open_cash_shift    -> if not has_any_permission(array['caixas.abrir'])      then raise ...
add_cash_movement -> if not has_any_permission(array['caixas.movimentacao']) then raise ...
close_cash_shift   -> if not has_any_permission(array['caixas.fechar'])    then raise ...
credit_partial_payment / reverse_partial_payment / reverse_paid_order / record_cash_expense -> idem
```

-- ...mas estas duas NAO tem guard nenhum, e sao security definer:

```
create or replace function public.create_order_and_credit_cash(...) ... security definer
create or replace function public.close_comanda_and_pay(...) ... security definer
grant execute on function public.create_order_and_credit_cash(...) to authenticated, anon; -- 1065
grant execute on function public.close_comanda_and_pay(...) to authenticated; -- 1068
...
```

Impacto

Cozinha/estoque registra vendas falsas e injeta valores no turno de caixa aberto, quebrando a conferência.

Sugestão de correção

Adicionar, no inicio de ambas (para o ramo nao-anonimo):

```
if auth.role() <> 'anon' and not public.has_any_permission(array[
  'pdv.finalizar_venda','mesas.fechar_comanda','online_menu.finalizar_pedido']) then
  raise exception 'Sem permissao para registrar venda.';
end if;
```

Manter o ramo anon (cardapio publico) como esta – ja e fortemente validado.

Critérios de aceite

- [] Sessão cozinha/estoque recebe exceção "Sem permissão" ao chamar create_order_and_credit_cash
- [] PDV (caixa) e fechamento de comanda (garçom/caixa) seguem funcionando
- [] Pedido público anônimo em /pedir continua funcionando
- [] Cobertura de teste para o guard nas duas RPCs

--- FIM ISSUE 4 ---

--- ISSUE 5 ---

[Segurança] Bucket product-images: escrita e exclusão sem escopo de dono nem permissão

****Labels:**** security, baixa

****Classificação:**** Baixa · Categoria 2 (Permissão definida no navegador)

Problema e por que é explorável

Qualquer funcionario autenticado pode sobrescrever ou apagar todos os objetos do bucket: fotos de produto, logo da empresa e capa do cardapio online (company_profile.logo_url / cover_url, que aparecem na pagina publica /pedir). Resultado: defacement do cardapio publico e perda de midia, feito por conta de baixo privilegio. A UI so oferece gestao de imagem a quem tem produtos.editar / empresa.editar_midia. O bucket e publico para leitura (por design), entao o dano e de integridade/disponibilidade.

Evidência

- `src/lib/storage.ts:54-64`

- `DEV\_DB\_SETUP.md:bloco SQL do bucket`

...

```
create policy "product_images_authenticated_write" on storage.objects
  for insert to authenticated with check (bucket_id = 'product-images');
create policy "product_images_authenticated_update" on storage.objects
  for update to authenticated using (bucket_id = 'product-images');
create policy "product_images_authenticated_delete" on storage.objects
  for delete to authenticated using (bucket_id = 'product-images');
-- sem owner = auth.uid(), sem has_any_permission(...)
...
```

Impacto

Defacement do cardápio público e perda de todas as imagens de produto por qualquer conta autenticada.

Sugestão de correção

Escopar update/delete por dono (owner = auth.uid()) e/ou por permissao.

Para a midia institucional (logo/capa), usar prefixo proprio (empresa/) e exigir empresa.editar_midia; para produtos/, exigir produtos.editar ou produtos.criar. O INSERT amplo pode continuar.

Critérios de aceite

- [] Funcionário sem produtos.editar não consegue deletar objeto em produtos/
- [] Funcionário sem empresa.editar_midia não consegue sobrescrever empresa/logo.* ou empresa/cover.*
- [] Upload de imagem de produto pela tela de Produtos continua funcionando

--- FIM ISSUE 5 ---

--- ISSUE 6 ---

[Segurança] Diretório supabase/.temp/ versionado (metadados de projeto e string de conexão do pooler)

****Labels:**** security, informativa

****Classificação:**** Informativa · Categoria 4 (Chaves expostas / arquivos sensíveis)

Problema e por que é explorável

Não há segredo real aqui: a pooler-url traz só usuário e host (sem senha), o project ref já vai embutido em VITE_SUPABASE_URL no bundle, e as versões de componentes são públicas. Mesmo assim, expõe organização, nome do projeto de dev e a topologia do banco sem necessidade, e indica que o .gitignore não está sendo respeitado para esse caminho (risco de, no futuro, entrar ali um arquivo que de fato seja sensível). Classificado como informativo.

Evidência

```
- `supabase/.temp/pooler-url:1`
- `supabase/.temp/linked-project.json:1`
- `\.gitignore:9`
```

...

git ls-files inclui:

```
supabase/.temp/pooler-url          -> postgresql://postgres.vxgkslytnofxtkhjpkxb@aws-0-us-west-2.pooler.supabase.com:5432/postgres
```

```
supabase/.temp/linked-project.json ->
```

```
{"ref":"vxgkslytnofxtkhjpkxb","name":"ampliechef_dev","organization_id":"ikrbfkaluvaemxzlrhbi",...}
```

```
supabase/.temp/project-ref, gotrue-version, postgres-version, storage-version, ...
```

.gitignore tem ".temp/" (linha 9), mas os arquivos foram adicionados antes da regra e continuam rastreados

...

Impacto

Baixo – expõe organização, nome do projeto de dev e topologia do banco; sinaliza .gitignore não aplicado nesse caminho.

Sugestão de correção

```
git rm -r --cached supabase/.temp && git commit.
```

Confirmar que .gitignore cobre supabase/.temp/ (a regra .temp/ já cobre; o problema é só o index/histórico).

Revisar histórico: git log --all -- supabase/.temp para garantir que nunca houve um .temp com access-token.

Critérios de aceite

- [] git ls-files | grep supabase/.temp não retorna nada
- [] supabase/.temp/ continua ignorado pelo git localmente
- [] Nenhum outro arquivo de credencial (.env, tokens) aparece em git ls-files

--- FIM ISSUE 6 ---